Greedy Algorithms

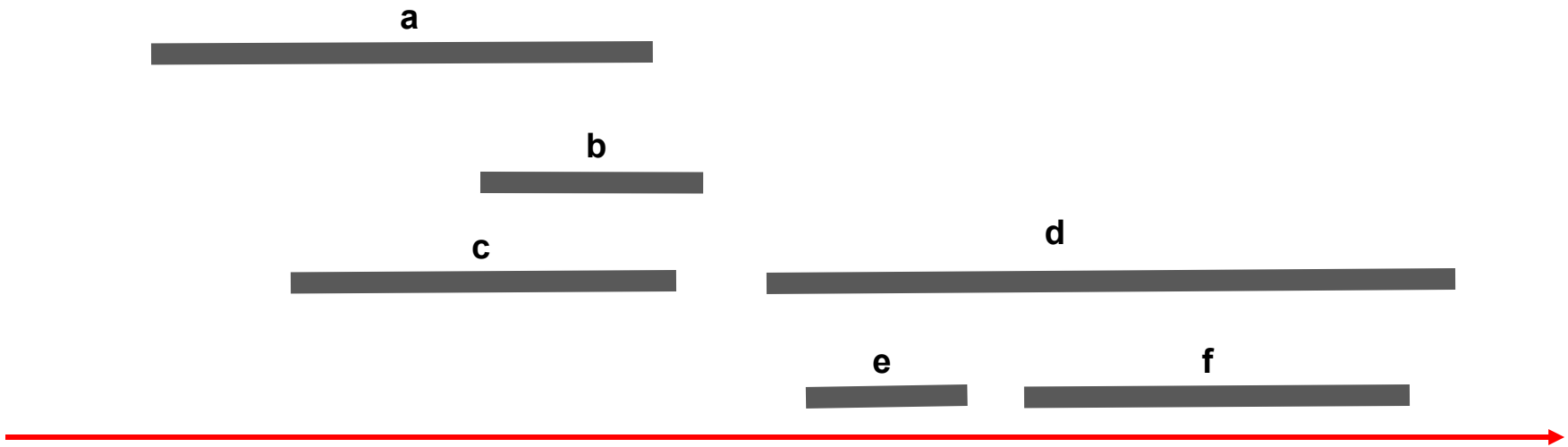
Greedy Algorithms

- Greedy algorithms are often used for solving **optimization problems**:
  - problems with many possible solutions. Each solution has a value and we wish to find an optimal solution: a solution with the optimal (maximum or minimum) value subject to some constraints:
    - Minimize the number of coins given the set of coins
    - Maximize the number of activities you can go, given that you cannot be at more than one event at a time
- Greedy Algorithms build solutions by choosing locally best optimal option at each step:
  - If the globally best solution is obtained by repeatedly choosing the locally best option the problem can be solved using greedy technique

Activity Selection/Interval Scheduling

Activity Selection/Interval scheduling

- **Input:** a list of activities $(s_1, f_1), (s_2, f_2), \ldots, (s_n, f_n)$
  - s_i: start time of activity i
  - f_i: finish time of activity i
- **Output:** maximize the **number** activities, such that there are no overlapping activity



Activity Selection: Brute Force

Brute-force technique:

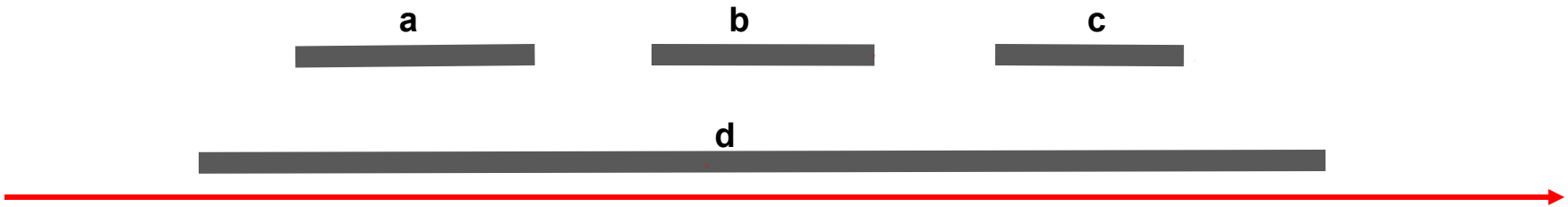
- Generate all subsets of activities
- for each subset of activities
  - Check if the activities are compatible (have overlap or not)
- Pick the subset with maximum number of elements and no overlap

Runtime: $\Omega(2^n)$

Activity Selection: Greedy Alg'm 1

Select the activity that start earliest

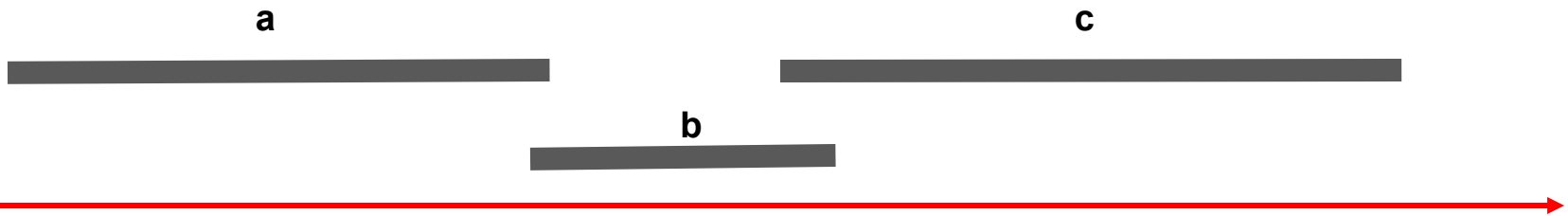
- sort the activities in order of start time. S=[]
- for i =1 to n
  - If activity i has no overlap with other activities in S
    - Add activity i to S
- return S
- Counter-example:
  - Algorithm 1 does not work. Optimal solution has 3 activities: a, b, c



Activity Selection: Greedy Alg'm 2

Select the activity that has the shortest duration

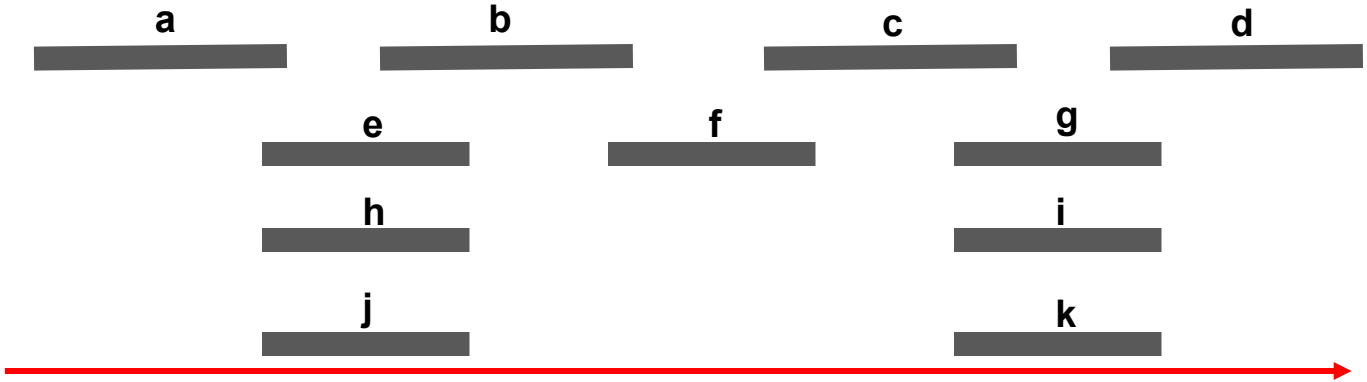
- sort the activities in order of f_i-s_i. S = []
- for i =1 to n
  - If activity i has no overlap with other activities in S
    - Add i to S
- return S
- Counter-example:
  - Algorithm 2 does not work, optimal solution: a,c



Activity Selection: Greedy Alg'm 3

Select the activity with the fewest number of overlaps

- sort the activities in order of their number of overlaps. S = []
- for i =1 to n
  - If activity i has no overlap with other activities in S
    - Add i to S
- return S
- Counter-example: Algorithm 3: **e, f, g** Optimal solution: **a, b, c, d**



Activity Selection: Greedy Alg'm 4: Optimal Solution

Select the activity with the earliest finish time

- Sort the activities in order of their finish time. S = []
- for i =1 to n
  - If activity i has no overlap with other activities in S
    - Add i to S
- return S

Local decision: Maximize the time left to satisfy other requests

Activity Selection: Greedy Alg'm 4: Runtime Analysis

A = List of all activities

Sort A into ascending order by finishing time → O(n log n)

S = []

for i=1 to n → This loop will iterate through all n intervals, visiting each exactly once for O(n) total iterations.

    If A[i] does not overlap S: → Is done by comparing the start time of activity i and finish time of the last activity added to S. The comparison takes O(1) time.

    S = S.append(A[i])

return S

Activity Selection: Alg'm 4: Proof of optimality: Notations

$\mathcal{G}$ = Greedy Solution (earliest finish time)

$\mathcal{O}$ = Optimal solution

f($\mathcal{G}$, i): the finish time of activity i in greedy solution: $\mathcal{G}$

f($\mathcal{O}$, i): the finish time of activity i in optimal solution: $\mathcal{O}$

Activity Selection: Alg'm 4: Proof of optimality

- To prove optimality of algorithm, we use "**greedy stays ahead**" argument
  - In each step greedy algorithm **does better than an optimal solution**
    - In each step greedy algorithm **chooses an activity that finishes before an activity chosen by an optimal solution**
      - Why this is better? Since it means greedy algorithm cannot select fewer activities compared to an optimal solution
        - the activity selected by the optimal solution is available for selection and the greedy cannot miss it in the next step.
  - Then use that to prove the algorithm produces an optimal solution

In other words, we need to prove f($\mathcal{G}$, k) ≤ f($\mathcal{O}$, k)

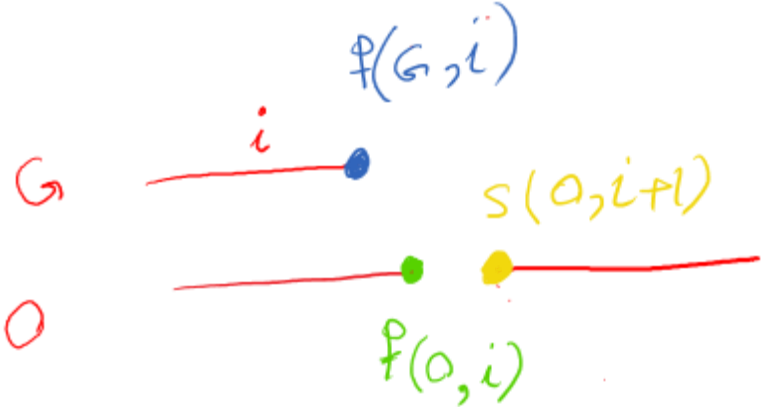
Proof of optimality: Alg'm 4: Greedy stays ahead

- **Theorem(k):** In step k, the greedy algorithm selects an activity that finishes no later than the activity chosen in step K by any optimal solution.
- **Proof by induction**
  - **Base case:** f($\mathcal{G}$, 1)≤ f($\mathcal{O}$, 1) : The greedy algorithm selects an activity with minimum finish time
  - **Induction hypothesis:** T(i) is True. f($\mathcal{G}$, i)≤ f($\mathcal{O}$, i)
  - **Induction step:** Showing T(i+1) is true: f($\mathcal{G}$, i+1) ≤ f($\mathcal{O}$, i+1)
    - f($\mathcal{G}$, i)≤ f($\mathcal{O}$, i) → i-th activity in $\mathcal{G}$ finishes before i-th activity in $\mathcal{O}$.
    - f($\mathcal{O}$, i) ≤ S($\mathcal{O}$, i+1):The (i+1)-st activity in $\mathcal{O}$ must start after the i-th activity in $\mathcal{O}$ ends
    - →f($\mathcal{G}$, i)≤ S($\mathcal{O}$, i+1): (i + 1)st activity in $\mathcal{O}$ must start after the ith activity in $\mathcal{G}$ finishes
    - → Therefore, when the greedy algorithm selects its (i+1)-st activity, the (i+1)-st activity in $\mathcal{O}$ must be there. The greedy algorithm selects the activity with lowest end time, therefore
    - **f($\mathcal{G}$, i+1) ≤ f($\mathcal{O}$, i+1)**

Proof of optimality: Greedy stays ahead

$f(G,i) \leq f(O,i) \leq s(O,i+1)$

induction hypothesis



$f(G,i) \leq s(O,i+1)$

when greedy selects $(i+1)$-th activity, the $(i+1)$-th activity in O is there

$f(G,i+1) \leq f(O,i+1)$

Proof of optimality

- Theorem: The greedy algorithm for activity selection produces an optimal schedule.
  - $|\mathcal{G}| \leq |\mathcal{O}|$ since $\mathcal{O}$ is optimal. To prove greedy algorithm is optimal we must show that $|\mathcal{G}| \geq |\mathcal{O}|$
- **Proof by contradiction:** Suppose by contradiction that the greedy algorithm is not optimal, $|\mathcal{G}| < |\mathcal{O}|$
  - Let k = $|\mathcal{G}|$.
  - $|\mathcal{G}| < |\mathcal{O}| \rightarrow$ f($\mathcal{G}$, k) ≤ f($\mathcal{O}$, k) < S($\mathcal{O}$, k+1): there is a (k + 1)st activity in $\mathcal{O}$ and its start time must be after f($\mathcal{O}$, k) and therefore after f($\mathcal{G}$, k).
  - Thus, after the greedy algorithm added its k-th activity to $\mathcal{G}$, the (k + 1)st activity would still be there. The (k+1)st activity has no overlap with greedy solution, and it should have been added to the greedy solution by the greedy algorithm →Contradiction
    - Greedy keeps selecting jobs until no more compatible jobs left.
    - $\rightarrow |\mathcal{G}| \geq |\mathcal{O}|$
  - → The greedy algorithm must be optimal: $|\mathcal{G}| == |\mathcal{O}|$

$|G| < |K| \Rightarrow f(G, k) \leq f(O, k) \leq S(O, k+1)$

$k = |G|$

proved earlier, greedy stays ahead

→ activity $k+1$ doesn't overlap with k-th activity

→ The greedy should have selected it, but if did not → contradiction